

ObjektinisProg

Sugeneruota Doxygen 1.17.0

1 ObjektinisProg	1
1.1 Naudojimosi instrukcija	1
1.2 Duomenų įvestis ir išvestis	1
1.2.1 Įvestis rankiniu būdu (1 parinktis)	1
1.2.2 Įvestis automatinio būdu (2 ir 3 parinktis)	1
1.2.3 Įvestis iš failo (4 parinktis)	2
1.2.4 Išvestis į ekraną (2 išvedimo būdas)	2
1.2.5 Išvestis į failą (1 ir 3 išvedimo būdas)	2
1.3 Perdengtų operatorių aprašas	2
1.3.1 <code>operator<<</code> — išvesties operatorius	2
1.3.2 <code>operator>></code> — įvesties operatorius	2
1.4 Kompiliavimas	2
1.4.1 Testų kompiliavimas ir paleidimas	3
1.4.2 Spartos benchmark	3
1.5 Klasės ir Struct Kompiliavimo flag'ų testas	3
1.5.1 100 000 studentų	3
1.5.2 1 000 000 studentų	3
1.6 Vienetų testai	3
1.6.1 Testų paskirtis	4
1.7 Spartos tyrimas	4
1.7.1 1 strategija	4
1.7.2 2 strategija	5
1.7.3 3 strategija	5
1.8 Išvados	6
1.9 Dokumentacija	6
1.9.1 Dokumentacijos generavimas	6
1.10 Vector konteineris (v3.0)	6
1.10.1 Implementuotos funkcijos	6
1.10.2 <code>push_back</code> spartos analizė	7
1.10.3 Atminties perskirstymai	7
1.11 Vector spartos analizė – pilna programa	7
1.11.1 100 000 studentų	7
1.11.2 1 000 000 studentų	7
1.11.3 10 000 000 studentų	8
1.12 Versijų istorija	8
2 Direktorijų hierarchija	9
2.1 Direktorijos	9
3 Hierarchijos Indeksas	11
3.1 Klasių hierarchija	11
4 Klasės Indeksas	13

4.1 Klasės	13
5 Failo Indeksas	15
5.1 Failai	15
6 Direktorijų dokumentacija	17
6.1 src Direktorijos aprašas	17
6.2 test Direktorijos aprašas	17
7 Klasės Dokumentacija	19
7.1 studentas Klasė	19
7.1.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	20
7.1.1.1 studentas() [1/4]	20
7.1.1.2 studentas() [2/4]	20
7.1.1.3 ~studentas()	20
7.1.1.4 studentas() [3/4]	20
7.1.1.5 studentas() [4/4]	20
7.1.2 Metodų Dokumentacija	20
7.1.2.1 addPaz()	20
7.1.2.2 clearPaz()	20
7.1.2.3 getEgz()	20
7.1.2.4 getGal()	21
7.1.2.5 getPavarde()	21
7.1.2.6 getPaz()	21
7.1.2.7 getRez()	21
7.1.2.8 getVardas()	21
7.1.2.9 med()	21
7.1.2.10 operator=() [1/2]	21
7.1.2.11 operator=() [2/2]	21
7.1.2.12 print()	21
7.1.2.13 setEgz()	21
7.1.2.14 setGal()	21
7.1.2.15 setPavarde()	21
7.1.2.16 setPaz()	22
7.1.2.17 setRez()	22
7.1.2.18 setVardas()	22
7.1.2.19 vid()	22
7.1.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija	22
7.1.3.1 operator<<	22
7.1.3.2 operator>>	22
7.2 Vector< T > Klasė Šablonas	22
7.2.1 Tipo Aprašymo Dokumentacija	24
7.2.1.1 const_iterator	24

7.2.1.2 <code>const_reverse_iterator</code>	24
7.2.1.3 <code>iterator</code>	24
7.2.1.4 <code>reverse_iterator</code>	24
7.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	24
7.2.2.1 <code>~Vector()</code>	24
7.2.2.2 <code>Vector()</code> [1/5]	24
7.2.2.3 <code>Vector()</code> [2/5]	24
7.2.2.4 <code>Vector()</code> [3/5]	24
7.2.2.5 <code>Vector()</code> [4/5]	24
7.2.2.6 <code>Vector()</code> [5/5]	24
7.2.3 Metodų Dokumentacija	25
7.2.3.1 <code>assign()</code> [1/3]	25
7.2.3.2 <code>assign()</code> [2/3]	25
7.2.3.3 <code>assign()</code> [3/3]	25
7.2.3.4 <code>at()</code> [1/2]	25
7.2.3.5 <code>at()</code> [2/2]	25
7.2.3.6 <code>back()</code> [1/2]	25
7.2.3.7 <code>back()</code> [2/2]	25
7.2.3.8 <code>begin()</code> [1/2]	25
7.2.3.9 <code>begin()</code> [2/2]	25
7.2.3.10 <code>capacity()</code>	25
7.2.3.11 <code>cbegin()</code>	26
7.2.3.12 <code>cend()</code>	26
7.2.3.13 <code>clear()</code>	26
7.2.3.14 <code>crbegin()</code>	26
7.2.3.15 <code>crend()</code>	26
7.2.3.16 <code>data()</code> [1/2]	26
7.2.3.17 <code>data()</code> [2/2]	26
7.2.3.18 <code>emplace()</code>	26
7.2.3.19 <code>emplace_back()</code>	26
7.2.3.20 <code>empty()</code>	26
7.2.3.21 <code>end()</code> [1/2]	26
7.2.3.22 <code>end()</code> [2/2]	27
7.2.3.23 <code>erase()</code> [1/2]	27
7.2.3.24 <code>erase()</code> [2/2]	27
7.2.3.25 <code>front()</code> [1/2]	27
7.2.3.26 <code>front()</code> [2/2]	27
7.2.3.27 <code>insert()</code> [1/2]	27
7.2.3.28 <code>insert()</code> [2/2]	27
7.2.3.29 <code>max_size()</code>	27
7.2.3.30 <code>operator"!="()</code>	27
7.2.3.31 <code>operator<()</code>	27

7.2.3.32 operator<=()	28
7.2.3.33 operator=() [1/3]	28
7.2.3.34 operator=() [2/3]	28
7.2.3.35 operator=() [3/3]	28
7.2.3.36 operator==()	28
7.2.3.37 operator>()	28
7.2.3.38 operator>=()	28
7.2.3.39 operator[]() [1/2]	28
7.2.3.40 operator[]() [2/2]	28
7.2.3.41 pop_back()	28
7.2.3.42 push_back() [1/2]	29
7.2.3.43 push_back() [2/2]	29
7.2.3.44 rbegin() [1/2]	29
7.2.3.45 rbegin() [2/2]	29
7.2.3.46 rend() [1/2]	29
7.2.3.47 rend() [2/2]	29
7.2.3.48 reserve()	29
7.2.3.49 resize()	29
7.2.3.50 shrink_to_fit()	29
7.2.3.51 size()	29
7.2.3.52 swap()	29
7.3 Zmogus Klasė	30
7.3.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	30
7.3.1.1 Zmogus() [1/2]	30
7.3.1.2 Zmogus() [2/2]	30
7.3.1.3 ~Zmogus()	30
7.3.2 Metodų Dokumentacija	30
7.3.2.1 getPavarde()	30
7.3.2.2 getVardas()	31
7.3.2.3 print()	31
7.3.2.4 setPavarde()	31
7.3.2.5 setVardas()	31
7.3.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija	31
7.3.3.1 operator<<	31
7.3.4 Atributų Dokumentacija	31
7.3.4.1 pavarde	31
7.3.4.2 vardas	31
8 Failo Dokumentacija	33
8.1 README.md Failo Nuoroda	33
8.2 src/main.cpp Failo Nuoroda	33
8.2.1 Funkcijos Dokumentacija	33

8.2.1.1 main()	33
8.3 test/main.cpp Failo Nuoroda	33
8.3.1 Funkcijos Dokumentacija	34
8.3.1.1 sukurtiStudenta()	34
8.3.1.2 TEST() [1/18]	34
8.3.1.3 TEST() [2/18]	34
8.3.1.4 TEST() [3/18]	34
8.3.1.5 TEST() [4/18]	34
8.3.1.6 TEST() [5/18]	34
8.3.1.7 TEST() [6/18]	34
8.3.1.8 TEST() [7/18]	35
8.3.1.9 TEST() [8/18]	35
8.3.1.10 TEST() [9/18]	35
8.3.1.11 TEST() [10/18]	35
8.3.1.12 TEST() [11/18]	35
8.3.1.13 TEST() [12/18]	35
8.3.1.14 TEST() [13/18]	35
8.3.1.15 TEST() [14/18]	35
8.3.1.16 TEST() [15/18]	35
8.3.1.17 TEST() [16/18]	35
8.3.1.18 TEST() [17/18]	36
8.3.1.19 TEST() [18/18]	36
8.4 src/mylib.cpp Failo Nuoroda	36
8.4.1 Funkcijos Dokumentacija	36
8.4.1.1 generuotiFaila()	36
8.4.1.2 generuotiPazymius()	36
8.4.1.3 generuotiViska()	36
8.4.1.4 getFile()	37
8.4.1.5 getInt()	37
8.4.1.6 iverstiRanka()	37
8.4.1.7 pagalGal()	37
8.4.1.8 pagalPavard()	37
8.4.1.9 pagalVard()	37
8.4.1.10 printRez()	37
8.4.1.11 randomstr()	37
8.4.1.12 skaitytiIsFailo()	37
8.4.1.13 skirstymas()	37
8.5 src/mylib.h Failo Nuoroda	38
8.5.1 Tipų apibrėžimų Dokumentacija	38
8.5.1.1 Studentai	38
8.5.2 Funkcijos Dokumentacija	38
8.5.2.1 generuotiFaila()	38

8.5.2.2 generuotiPazymius()	38
8.5.2.3 generuotiViska()	39
8.5.2.4 getFile()	39
8.5.2.5 getInt()	39
8.5.2.6 iverstiRanka()	39
8.5.2.7 pagalGal()	39
8.5.2.8 pagalPavard()	39
8.5.2.9 pagalVard()	39
8.5.2.10 printRez()	39
8.5.2.11 randomstr()	39
8.5.2.12 rusiuoti()	39
8.5.2.13 skaitytiIsFailo()	40
8.5.2.14 skirstymas()	40
8.6 mylib.h	40
8.7 src/vector.h Failo Nuoroda	42
8.7.1 Funkcijos Dokumentacija	42
8.7.1.1 swap()	42
8.8 vector.h	42
8.9 test/benchmark.cpp Failo Nuoroda	46
8.9.1 Tipų apibrėžimų Dokumentacija	46
8.9.1.1 ms	46
8.9.2 Funkcijos Dokumentacija	46
8.9.2.1 benchmark_push_back()	46
8.9.2.2 count_reallocations()	46
8.9.2.3 main()	47
8.10 test/benchmark_full.cpp Failo Nuoroda	47
8.10.1 Funkcijos Dokumentacija	47
8.10.1.1 benchmark()	47
8.10.1.2 main()	47
8.10.1.3 ms()	47
8.10.1.4 rusiuoti_bench()	47
8.10.1.5 skaiciuoti()	48
8.10.1.6 skaityti()	48
8.10.1.7 skirstyti_bench()	48
8.11 test/vector_test.cpp Failo Nuoroda	48
8.11.1 Funkcijos Dokumentacija	49
8.11.1.1 TEST() [1/33]	49
8.11.1.2 TEST() [2/33]	49
8.11.1.3 TEST() [3/33]	49
8.11.1.4 TEST() [4/33]	49
8.11.1.5 TEST() [5/33]	49
8.11.1.6 TEST() [6/33]	49

8.11.1.7 TEST() [7/33]	49
8.11.1.8 TEST() [8/33]	49
8.11.1.9 TEST() [9/33]	50
8.11.1.10 TEST() [10/33]	50
8.11.1.11 TEST() [11/33]	50
8.11.1.12 TEST() [12/33]	50
8.11.1.13 TEST() [13/33]	50
8.11.1.14 TEST() [14/33]	50
8.11.1.15 TEST() [15/33]	50
8.11.1.16 TEST() [16/33]	50
8.11.1.17 TEST() [17/33]	50
8.11.1.18 TEST() [18/33]	50
8.11.1.19 TEST() [19/33]	51
8.11.1.20 TEST() [20/33]	51
8.11.1.21 TEST() [21/33]	51
8.11.1.22 TEST() [22/33]	51
8.11.1.23 TEST() [23/33]	51
8.11.1.24 TEST() [24/33]	51
8.11.1.25 TEST() [25/33]	51
8.11.1.26 TEST() [26/33]	51
8.11.1.27 TEST() [27/33]	51
8.11.1.28 TEST() [28/33]	51
8.11.1.29 TEST() [29/33]	52
8.11.1.30 TEST() [30/33]	52
8.11.1.31 TEST() [31/33]	52
8.11.1.32 TEST() [32/33]	52
8.11.1.33 TEST() [33/33]	52
Rodyklė	53

skyrius 1

ObjektinisProg

1.1 Naudojimosi instrukcija

Funkcijos:

- Pasirinktinai išvestis į failą arba į terminalą.
- Pasirinktinai galutinio rezultato skaičiavimas, remiantis vidurkiu arba mediana.
- Rūšiavimas pasirinktinu būdu.
- Studentų skirstymas į „nevykelius“ (galutinis < 5) ir „nerdus“ (galutinis ≥ 5).

1 parinktis – rankinis duomenų įvedimas: studento vardo, pavardės, namų darbų rezultatų, egzamino rezultato.
2 parinktis – pusiau rankinis įvedimas: studento vardo, pavardės įvedimas, rezultatų generavimas.
3 parinktis – automatinis studentų vardų, pavardžių, rezultatų generavimas, jų apdorojimas ir išvedimas.
4 parinktis – skaitymas iš pasirinkto failo, rūšiavimas pasirinktinu būdu, duomenų apdorojimas ir išvedimas.
5 parinktis – studentų failų generavimas: studentų, namų darbų kiekio pasirinkimas ir išvedimas į studentų failą.
6 parinktis – programos nutraukimas.

Pasirinkus 1–4 parinktį, programa toliau klausia:

Rūšiavimo kriterijus	- pagal vardą, pavardę arba galutinį balą.
Skaiciavimo būdas	- galutinis balas skaičiuojamas vidurkiu arba mediana.
Išvedimo būdas	- į failą, į ekraną arba skirstymas į „nevykelių“ ir „nerdų“ failus.

1.2 Duomenų įvestis ir išvestis

1.2.1 Įvestis rankiniu būdu (1 parinktis)

Vartotojas įveda kiekvieno studento duomenis ranka. Programa prašo vardo, pavardės, namų darbų pažymių (baigiama įvedant -1) ir egzamino pažymio. Naudojama `getInt()` funkcija su validacija — neteisingi įvedimai atmami.

```
Įrašykite studento vardą (arba -1 baigti): Jonas
Įrašykite studento pavardę: Jonaitis
Įrašykite studento nd pažymį (arba -1 baigti): 8
Įrašykite studento nd pažymį (arba -1 baigti): 9
Įrašykite studento nd pažymį (arba -1 baigti): -1
Įrašykite studento egzamino pažymį: 10
```

1.2.2 Įvestis automatinio būdu (2 ir 3 parinktis)

2 parinktis — vartotojas įveda vardą ir pavardę, pažymiai generuojami atsitiktinai.

3 parinktis — viskas generuojama automatiškai: vardai, pavardės, pažymiai.

1.2.3 Įvestis iš failo (4 parinktis)

Programa nuskaito studentų duomenis iš tekstinio failo. Failo formatas:

Vardas	Pavarde	ND1	ND2	... Egz.
Vardas1	Pavardel	7	8	... 9

Kiekviena eilutė nuskaitoma per `operator>>`, kuris automatiškai išskiria vardą, pavardę, namų darbų pažymius ir egzaminą.

1.2.4 Išvestis į ekraną (2 išvedimo būdas)

Rezultatai spausdinami į terminalą per `operator<<`:

Vardas	Pavardė	Galutinis (Vid.)
-----	-----	-----
Jonas	Jonaitis	8.80

1.2.5 Išvestis į failą (1 ir 3 išvedimo būdas)

1 būdas — visi studentai išvedami į `isvedimas.txt`.

3 būdas — studentai skirstomi: nevykeliai į `nevykeliai.txt`, nerds į `nerds.txt`.

1.3 Perdengtų operatorių aprašas

`studentas` klasėje perdengiami du srautų operatoriai: `operator>>` (įvestis) ir `operator<<` (išvestis). Abu deklaruojami kaip `friend` funkcijos, nes turi tiesiogiai pasiekti `private` klasės laukus.

1.3.1 `operator<<` — išvesties operatorius

```
friend std::ostream& operator<<(std::ostream& out, const studentas& s);
```

Išveda studento vardą, pavardę ir galutinį balą suformatuotai. Grąžina `out`, kad būtų galimas grandininis išvedimas.

Naudojamas `printRez()` funkcijoje išvedant visą studentų sąrašą:

```
for (const auto& s : A)
    out << s << '\n';
```

Taip pat galima naudoti tiesiogiai:

```
studentas s("Jonas", "Jonaitis", {8, 9}, 10);
s.setGal(8.80);
std::cout << s;
// Jonas          Jonaitis          8.80
```

1.3.2 `operator>>` — įvesties operatorius

```
friend std::istream& operator>>(std::istream& in, studentas& s);
```

Nuskaityto vardą, pavardę, visus skaičius — paskutinis laikomas egzamino pažymiu, likusieji — namų darbų pažymiais. Grąžina `in` grandininiam skaitymui.

Naudojamas `skaitytiIsFailo()` funkcijoje:

```
std::stringstream iss(line);
iss >> temp;
```

Taip pat galima naudoti tiesiogiai su `std::stringstream`:

```
studentas s;
std::stringstream iss("Jonas Jonaitis 8 9 10");
iss >> s;
// s.getVardas() == "Jonas", s.getEgz() == 10, paz == {8, 9}
```

1.4 Kompiliavimas

Reikalavimai: CMake 3.14, MinGW (Windows) arba GCC (Linux).

```
mkdir build
cd build
cmake .. -G "MinGW Makefiles"          # numatytasis - Vector<T>
cmake --build .
```

Konteinerio pasirinkimas:

```
cmake .. -G "MinGW Makefiles" -DCONTAINER=myvector # Vector<T> (numatytasis)
cmake .. -G "MinGW Makefiles" -DCONTAINER=vector   # std::vector
cmake .. -G "MinGW Makefiles" -DCONTAINER=list     # std::list
cmake .. -G "MinGW Makefiles" -DCONTAINER=deque    # std::deque
```

1.4.1 Testų kompiliavimas ir paleidimas

```
cmake .. -G "MinGW Makefiles" -DBUILD_TESTS=ON
cmake --build .

./tests          # studentas klasės testai (18 testų)
./vector_tests   # Vector<T> klasės testai (35 testai)
```

1.4.2 Spartos benchmark

```
cmake .. -G "MinGW Makefiles" -DBUILD_BENCHMARK=ON -DBUILD_BENCHMARK_FULL=ON
cmake --build .

./benchmark      # push_back spartos testas
./benchmark_full # pilnas programos spartos testas
```

1.5 Klasės ir Struct Kompiliavimo flag'ų testas

1.5.1 100 000 studentų

Optimizavimas	Versija	Vykdyto laikas (ms)	Failo dydis (ms)
O1	struct	873.15	1035
	class	657.62	1029
O2	struct	864.67	1036
	class	653.08	1026
O3	struct	854.61	1055
	class	641.19	1043

1.5.2 1 000 000 studentų

Optimizavimas	Versija	Vykdyto laikas (ms)	Failo dydis (ms)
O1	struct	9623.41	1035
	class	6617.74	1029
O2	struct	9543.10	1036
	class	6474.63	1026
O3	struct	9424.76	1055
	class	6518.39	1043

Rezultatai rodo, kad class visais atvejais veikė greičiau nei struct. Didinant optimizavimo lygį (O1 → O3), vykdymo laikas mažėjo, o failo dydis didėjo.

1.6 Vientų testai

Vientų testai ([test/main.cpp](#)) realizuoti naudojant **Google Test** karkasą. Testai tikrina pagrindinį [studentas](#) klasės funkcionalumą ir Rule of Five realizaciją. GTest atsisiunčiamas automatiškai per CMake `FetchContent` — rankinio diegimo nereikia.

Testuojamos šios dalys:

- Numatytasis konstruktorius.
- Kopijavimo konstruktorius.
- Kopijavimo priskyrimo operatorius.

- Perkėlimo konstruktorius.
- Perkėlimo priskyrimo operatorius.
- `vid()` ir `med()` funkcijos (įskaitant kraštinį atvejį — tuščias pažymių sąrašas).
- Galutinio balo skaičiavimas vidurkiu ir mediana.
- `skirstymas()` — teisingas skaidymas, visi vykeliai, visi nevykeliai, tuščias konteineris.
- `printRez()` — išvesties patikrinimas per `std::ostringstream`.

Sėkmingai praėjus testams išvedama:

```
[=====] Running 18 tests from 4 test suites.
[-----] 5 tests from StudentasKonstruktorius
[ RUN    ] StudentasKonstruktorius.Numatytasis
[       OK ] StudentasKonstruktorius.Numatytasis
...
[=====] 18 tests from 4 test suites ran.
[ PASSED ] 18 tests.
```

1.6.1 Testų paskirtis

Testai skirti užtikrinti, kad:

- Rule of Five metodai veikia korektiškai.
- Objektų kopijavimas nesukelia bendrų duomenų problemų.
- Perkėlus objektą jo būsena išlieka validi.
- Vidurkio ir medianos skaičiavimai grąžina teisingus rezultatus.
- `skirstymas()` teisingai atskiria nevykelius visais kraštiniais atvejais.

1.7 Spartos tyrimas

Tyrimui naudojami prieš tai sugeneruoti failai. Bandyti visi rūšiavimo ir galutinio skaičiavimo būdai, rezultatai išreikšti vidurkiu.

Specs: AMD Ryzen 5 3600 · HyperX DDR4 16GB · SSD 970 M.2 250GB

1.7.1 1 strategija

Bendro studentai konteinerio skaidymas į du naujus to paties tipo konteinerius: „nevykeliai“ ir „nerds“. Tokiu būdu tas pats studentas yra dvejuose konteineriuose: bendrame studentai ir viename iš suskaidytų.

Studentų kiekis	Konteineris	Nuskaitymas (ms)	Rūšiavimas (ms)	Skirstymas (ms)
1 000	vector	2.53	0.18	0.09
	deque	1.74	0.21	0.08
	list	2.07	0.09	0.11
10 000	vector	45.12	26.47	0.43
	deque	41.61	29.34	0.29
	list	49.38	13.02	0.37
100 000	vector	439.84	363.55	16.28
	deque	441.07	404.92	15.83
	list	451.29	171.84	24.51
1 000 000	vector	4397.41	4868.73	192.74
	deque	4418.56	5341.08	169.17

Studentų kiekis	Konteineris	Nuskaitymas (ms)	Rūšiavimas (ms)	Skirstymas (ms)
	list	4454.83	2207.16	220.39
10 000 000	vector	44284.↵ 6	62254.37	1880.62
	deque	44514.↵ 8	69897.44	1865.93
	list	44742.↵ 3	28843.91	2444.18

1.7.2 2 strategija

Bendro studentų konteinerio skaidymas panaudojant tik vieną naują konteinerį: „nevykeliai“. Tokiu būdu, jei studentas yra nevykelis, jį turime įkelti į naująjį „nevykelių“ konteinerį ir ištrinti iš bendro studentų konteinerio. Po šio žingsnio studentai konteineryje liks vien tik nerds.

Studentų kiekis	Konteineris	Nuskaitymas (ms)	Rūšiavimas (ms)	Skirstymas (ms)
1 000	vector	4.67	2.00	0.36
	deque	4.32	0.67	0.00
	list	5.67	0.99	0.33
10 000	vector	43.85	28.67	2.00
	deque	49.99	29.15	0.00
	list	45.86	14.33	0.99
100 000	vector	427.02	377.19	12.96
	deque	435.62	246.08	13.86
	list	450.60	186.96	21.33
1 000 000	vector	4294.58	5088.73	141.03
	deque	4357.09	5189.21	161.02
	list	4516.69	2378.67	259.93
10 000 000	vector	45101.00	64604.73	1559.45
	deque	42853.97	66047.37	1807.13
	list	45069.80	30505.13	2956.44

1.7.3 3 strategija

Tas pats kaip 2 strategija, tačiau skirstymui naudojamas `std::partition` — elementai perstumiami vietoje (in-place) vienu perėjimu, po to nevykeliai nukopijuojami į naują konteinerį ir ištrinami iš bendro. Skirtingai nuo 2 strategijos, kuri naudoja `copy_if` ir `remove_if` (du perėjimai), `partition` tai atlieka vienu perėjimu.

Studentų kiekis	Konteineris	Nuskaitymas (ms)	Rūšiavimas (ms)	Skirstymas (ms)
1 000	vector	2.57	0.53	0.00
	deque	8.10	0.00	0.00
	list	9.97	0.00	0.00
10 000	vector	41.54	21.68	0.00
	deque	43.23	20.37	0.00

Studentų kiekis	Konteineris	Nuskaitymas (ms)	Rūšiavimas (ms)	Skirstymas (ms)
	list	45.32	16.32	0.00
100 000	vector	444.37	280.55	3.97
	deque	428.07	305.00	3.35
	list	424.28	186.82	8.17
1 000 000	vector	4453.88	3671.65	26.63
	deque	4267.51	4031.19	65.79
	list	4239.82	2598.73	100.15
10 000 000	vector	44772.97	40601.10	234.93
	deque	42661.50	48562.40	695.92
	list	42550.20	33182.10	1073.19

1.8 Išvados

Nuskaitymo ir rūšiavimo rezultatai išlieka panašūs visose trijose strategijose, kaip ir tikėtasi — skiriasi tik skirstymo dalis. Aiškiausiai tai matyti lyginant 2 ir 3 strategiją: `partition` (3 strat.) skirstymas yra žymiai greitesnis už `copy_if + remove_if` (2 strat.), nes atlieka tik vieną perėjimą per konteinerį vietoj dviejų. Skirtumas ypač ryškus didesniuose failuose — pvz., 10M studentų su `vector`: 2 strategija ~1559 ms, 3 strategija ~235 ms.

1.9 Dokumentacija

Projekto dokumentacija sugeneruota naudojant **Doxygen** ir prieinama `docs/` kataloge.

- `docs/html/index.html` — naršyklėje peržiūrima HTML dokumentacija.
- `docs/latex/refman.pdf` — sukompiliuotas PDF.

1.9.1 Dokumentacijos generavimas

`doxygen Doxyfile`

1.10 Vector konteineris (v3.0)

Vietoje `std::vector` naudojamas savarankiškai sukurtas `Vector<T>` šabloninis konteineris, dengiąs 80% `std::vector` funkcijų.

1.10.1 Implementuotos funkcijos

Kategorija	Funkcijos
Konstruktoriai	<code>Vector()</code> , <code>Vector(n, val)</code> , <code>Vector(init_list)</code> , <code>copy</code> , <code>move</code> , <code>operator=</code>
Capacity	<code>size()</code> , <code>capacity()</code> , <code>empty()</code> , <code>reserve()</code> , <code>shrink_to_fit()</code> , <code>max_size()</code>
Element access	<code>operator[]</code> , <code>at()</code> , <code>front()</code> , <code>back()</code> , <code>data()</code>
Modifiers	<code>push_back()</code> , <code>pop_back()</code> , <code>insert()</code> , <code>erase()</code> , <code>clear()</code> , <code>resize()</code> , <code>emplace_back()</code> , <code>assign()</code> , <code>swap()</code>
Iteratoriai	<code>begin()</code> , <code>end()</code> , <code>cbegin()</code> , <code>cend()</code> , <code>rbegin()</code> , <code>rend()</code>
Operatoriai	<code>==</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code>

1.10.2 push_back spartos analizė

Lyginamas `std::vector` ir `Vector` užpildymo laikas naudojant `push_back()` (5 paleidimų vidurkis, -O2).

Specs: AMD Ryzen 5 3600 · HyperX DDR4 16GB · SSD 970 M.2 250GB

Elementų sk.	std::vector (ms)	Vector (ms)	Santykis
10 000	0.200	0.000	0.00
100 000	1.400	1.000	0.71
1 000 000	14.575	8.802	0.60
10 000 000	149.533	120.233	0.80
100 000 000	1396.726	1146.366	0.82

`Vector` pasirodo lygiavertiškai arba greičiau – tai laukiamas rezultatas, nes implementacija naudoja tą pačią `capacity * 2` strategiją, tačiau be kai kurių `std::vector` papildomų patikrinimų.

1.10.3 Atminties perskirstymai

Užpildant 100 000 000 elementų `push_back()` funkcija:

Konteineris	Perskirstymų sk.
std::vector	28
Vector	28

Abu konteineriai naudoja `capacity * 2` strategiją, todėl perskirstymų skaičius sutampa. Perskirstymas įvyksta kai `capacity() == size()`.

1.11 Vector spartos analizė – pilna programa

Lyginamas `std::vector` ir `Vector<T>` veikimas naudojant realius studentų duomenų failus. Matuojamos visos operacijos: nuskaitymas, skaičiavimai, rūšiavimas ir skirstymas.

Specs: AMD Ryzen 5 3600 · HyperX DDR4 16GB · SSD 970 M.2 250GB

Kompiliavimas: -O2

1.11.1 100 000 studentų

Konteineris	Nuskaitymas (ms)	Skaičiavimai (ms)	Rūšiavimas (ms)	Skirstymas (ms)	Iš viso (ms)
std::vector	518.13	2.00	35.01	4.00	559.14
Vector	468.62	2.00	29.01	7.00	506.63

1.11.2 1 000 000 studentų

Konteineris	Nuskaitymas (ms)	Skaičiavimai (ms)	Rūšiavimas (ms)	Skirstymas (ms)	Iš viso (ms)
std::vector	4819.08	25.01	373.08	43.00	5260.17
Vector	4623.38	25.01	401.09	79.02	5128.49

1.11.3 10 000 000 studentų

Konteineris	Nuskaitymas (ms)	Skaiciavimai (ms)	Rūšiavimas (ms)	Skirstymas (ms)	Iš viso (ms)
<code>std::vector</code>	48743.07	248.06	4624.06	416.10	54031.28
<code>Vector</code>	47912.78	260.25	4675.22	715.88	53564.13

`Vector<T>` bendras veikimas yra lygiavertis `std::vector` — nuskaitymas ir rūšiavimas net šiek tiek greitesni dėl paprastesnės implementacijos. Skirstymas lėtesnis dėl `assign` su `make_move_iterator` — tai laukiamas rezultatas, nes `std::vector` turi optimizuotą šio metodo realizaciją.

1.12 Versijų istorija

Versija	Pakeitimai
v0.1	Pradinis variantas. Duomenų įvedimas ranka, C masyvo ir <code>std::vector</code> realizacijos, galutinio balo skaičiavimas vidurkiu arba mediana, išvedimas į ekraną.
v0.2	Pridėtas duomenų nuskaitymas iš failo, išvedimas į failą, rūšiavimas pagal vardą, pavardę arba galutinį balą. Projektas išskaidytas į kelis failus.
v0.3	Funkcijos perkeltos į antraštinį (.h) ir realizacijos (.cpp) failus. Pridėtas klaidų gaudymas (exception handling).
v0.4	Pridėtas failų generavimas, studentų skirstymas į „nevykelius“ ir „nerdus“, programos spartos tyrimas su 5 skirtingo dydžio failais.
v1.0	Trys atskiros realizacijos (vector, list, deque). Išbandytos 3 skirstymo strategijos. Pridėtas CMakeLists.txt.
v1.2	struct <code>studentas</code> pertvarkyta į class <code>studentas</code> su private laukais, getteriais ir setteriais. Realizuoti visi Rule of Five metodai (destruktorius, kopijavimo ir perkėlimo konstruktoriai, kopijavimo ir perkėlimo priskyrimo operatoriai). Perdengiami <code>operator<<</code> ir <code>operator>></code> įvesties/išvesties operatoriai. Pridėti vienetų testai (test.cpp).
v1.5	Pridėta abstrakti klasė <code>Zmogus</code> , iš kurios išvedama klasė <code>Studentas</code>
v2.0	Bendras src/ kodas visiems konteineriams — konteineris pasirenkamas per <code>-DCONTAINER=CMake</code> flagą. Vienetų testai perkelti į Google Test karkasą (18 testų). Pridėta Doxygen dokumentacija (HTML + PDF).
v3.0	Sukurtas savarankiškas <code>Vector<T></code> konteineris, dengiantis 80% <code>std::vector</code> metodų. Atlikta <code>push_back</code> spartos analizė ir atminties perskirstymų palyginimas. Programa integruota su <code>Vector</code> vietoje <code>std::vector</code> .

skyrius 2

Direktorių hierarchija

2.1 Direktorijos

src	17
main.cpp	33
mylib.cpp	36
mylib.h	38
vector.h	42
test	17
benchmark.cpp	46
benchmark_full.cpp	47
main.cpp	33
vector_test.cpp	48

skyrius 3

Hierarchijos Indeksas

3.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Vector< T >	22
Zmogus	30
studentas	19

skyrius 4

Klasės Indeksas

4.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

studentas	19
Vector< T >	22
Zmogus	30

skyrius 5

Failo Indeksas

5.1 Failai

Visų failų sąrašas su trumpais aprašymais:

src/main.cpp	33
src/mylib.cpp	36
src/mylib.h	38
src/vector.h	42
test/benchmark.cpp	46
test/benchmark_full.cpp	47
test/main.cpp	33
test/vector_test.cpp	48

skyrius 6

Direktorių dokumentacija

6.1 src Direktorijos aprašas

Failai

- failas [main.cpp](#)
- failas [mylib.cpp](#)
- failas [mylib.h](#)
- failas [vector.h](#)

6.2 test Direktorijos aprašas

Failai

- failas [benchmark.cpp](#)
- failas [benchmark_full.cpp](#)
- failas [main.cpp](#)
- failas [vector_test.cpp](#)

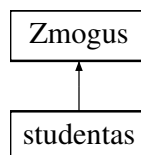
skyrius 7

Klasės Dokumentacija

7.1 studentas Klasė

```
#include <mylib.h>
```

Paveldimumo diagrama studentas:



Vieši Metodai

- `studentas ()`
- `studentas (const std::string &v, const std::string &p, const std::vector< int > &pazymiai, int e)`
- `~studentas ()` override
- `studentas (const studentas &other)`
- `studentas & operator= (const studentas &other)`
- `studentas (studentas &&other) noexcept`
- `studentas & operator= (studentas &&other) noexcept`
- `std::string getVardas ()` const override
- `std::string getPavarde ()` const override
- `void setVardas (const std::string &v)` override
- `void setPavarde (const std::string &p)` override
- `void print (std::ostream &out)` const override
- `std::vector< int > getPaz ()` const
- `int getEgz ()` const
- `double getRez ()` const
- `double getGal ()` const
- `void setPaz (const std::vector< int > &p)`
- `void addPaz (int p)`
- `void clearPaz ()`
- `void setEgz (int e)`
- `void setRez (double r)`
- `void setGal (double g)`
- `double vid ()` const
- `double med ()` const

Vieši Metodai inherited from **Zmogus**

- **Zmogus** ()
- **Zmogus** (const std::string &v, const std::string &p)
- virtual ~**Zmogus** ()

Draugai

- std::ostream & **operator<<** (std::ostream &out, const **studentas** &s)
- std::istream & **operator>>** (std::istream &in, **studentas** &s)

Additional Inherited Members

Apsaugoti Atributai inherited from **Zmogus**

- std::string **vardas**
- std::string **pavarde**

7.1.1 Konstruktorius ir Destruktorius Dokumentacija

7.1.1.1 **studentas()** [1/4]

```
studentas::studentas () [inline]
```

7.1.1.2 **studentas()** [2/4]

```
studentas::studentas (  
    const std::string & v,  
    const std::string & p,  
    const std::vector< int > & pazymiai,  
    int e) [inline]
```

7.1.1.3 ~**studentas()**

```
studentas::~~studentas () [inline], [override]
```

7.1.1.4 **studentas()** [3/4]

```
studentas::studentas (  
    const studentas & other) [inline]
```

7.1.1.5 **studentas()** [4/4]

```
studentas::studentas (  
    studentas && other) [inline], [noexcept]
```

7.1.2 Metodų Dokumentacija

7.1.2.1 **addPaz()**

```
void studentas::addPaz (  
    int p) [inline]
```

7.1.2.2 **clearPaz()**

```
void studentas::clearPaz () [inline]
```

7.1.2.3 **getEgz()**

```
int studentas::getEgz () const [inline]
```

7.1.2.4 getGal()

```
double studentas::getGal () const [inline]
```

7.1.2.5 getPavarde()

```
std::string studentas::getPavarde () const [inline], [override], [virtual]  
Realizuoja Zmogus.
```

7.1.2.6 getPaz()

```
std::vector< int > studentas::getPaz () const [inline]
```

7.1.2.7 getRez()

```
double studentas::getRez () const [inline]
```

7.1.2.8 getVardas()

```
std::string studentas::getVardas () const [inline], [override], [virtual]  
Realizuoja Zmogus.
```

7.1.2.9 med()

```
double studentas::med () const [inline]
```

7.1.2.10 operator=() [1/2]

```
studentas & studentas::operator= (  
    const studentas & other) [inline]
```

7.1.2.11 operator=() [2/2]

```
studentas & studentas::operator= (  
    studentas && other) [inline], [noexcept]
```

7.1.2.12 print()

```
void studentas::print (  
    std::ostream & out) const [inline], [override], [virtual]  
Realizuoja Zmogus.
```

7.1.2.13 setEgz()

```
void studentas::setEgz (  
    int e) [inline]
```

7.1.2.14 setGal()

```
void studentas::setGal (  
    double g) [inline]
```

7.1.2.15 setPavarde()

```
void studentas::setPavarde (  
    const std::string & p) [inline], [override], [virtual]  
Realizuoja Zmogus.
```

7.1.2.16 setPaz()

```
void studentas::setPaz (
    const std::vector< int > & p) [inline]
```

7.1.2.17 setRez()

```
void studentas::setRez (
    double r) [inline]
```

7.1.2.18 setVardas()

```
void studentas::setVardas (
    const std::string & v) [inline], [override], [virtual]
```

Realizuoja [Zmogus](#).

7.1.2.19 vid()

```
double studentas::vid () const [inline]
```

7.1.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija

7.1.3.1 operator<<

```
std::ostream & operator<< (
    std::ostream & out,
    const studentas & s) [friend]
```

7.1.3.2 operator>>

```
std::istream & operator>> (
    std::istream & in,
    studentas & s) [friend]
```

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [src/mylib.h](#)

7.2 Vector< T > Klasė Šablonas

```
#include <vector.h>
```

Vieši Tipai

- using [iterator](#) = T*
- using [const_iterator](#) = const T*
- using [reverse_iterator](#) = std::reverse_iterator<[iterator](#)>
- using [const_reverse_iterator](#) = std::reverse_iterator<[const_iterator](#)>

Vieši Metodai

- [~Vector](#) ()
- [size_t size](#) () const
- [size_t capacity](#) () const
- bool [empty](#) () const
- void [push_back](#) (const T &val)
- T & [operator\[\]](#) (size_t i)
- const T & [operator\[\]](#) (size_t i) const
- [Vector](#) ()
- [Vector](#) (size_t n, const T &val=T())

- `Vector` (`std::initializer_list< T > il`)
- `Vector` (`const Vector &other`)
- `Vector` (`Vector &&other`) `noexcept`
- `Vector & operator=` (`const Vector &other`)
- `Vector & operator=` (`Vector &&other`) `noexcept`
- `Vector & operator=` (`std::initializer_list< T > il`)
- `iterator begin` ()
- `iterator end` ()
- `const_iterator begin` () `const`
- `const_iterator end` () `const`
- `const_iterator cbegin` () `const`
- `const_iterator cend` () `const`
- `reverse_iterator rbegin` ()
- `reverse_iterator rend` ()
- `const_reverse_iterator rbegin` () `const`
- `const_reverse_iterator rend` () `const`
- `const_reverse_iterator crbegin` () `const`
- `const_reverse_iterator crend` () `const`
- `void reserve` (`size_t newCap`)
- `void shrink_to_fit` ()
- `size_t max_size` () `const`
- `T & at` (`size_t i`)
- `const T & at` (`size_t i`) `const`
- `T & front` ()
- `const T & front` () `const`
- `T & back` ()
- `const T & back` () `const`
- `T * data` ()
- `const T * data` () `const`
- `void clear` ()
- `void pop_back` ()
- `iterator insert` (`const_iterator pos`, `const T &val`)
- `iterator insert` (`const_iterator pos`, `size_t count`, `const T &val`)
- `iterator erase` (`const_iterator pos`)
- `iterator erase` (`const_iterator first`, `const_iterator last`)
- `void resize` (`size_t newSize`, `const T &val=T()`)
- `template<typename... Args>`
`iterator emplace` (`const_iterator pos`, `Args &&... args`)
- `template<typename... Args>`
`void emplace_back` (`Args &&... args`)
- `void push_back` (`T &&val`)
- `void assign` (`size_t count`, `const T &val`)
- `void assign` (`std::initializer_list< T > il`)
- `void swap` (`Vector &other`) `noexcept`
- `bool operator==` (`const Vector &other`) `const`
- `bool operator!=` (`const Vector &other`) `const`
- `bool operator<` (`const Vector &other`) `const`
- `bool operator<=` (`const Vector &other`) `const`
- `bool operator>` (`const Vector &other`) `const`
- `bool operator>=` (`const Vector &other`) `const`
- `template<typename InputIt>`
`void assign` (`InputIt first`, `InputIt last`)

7.2.1 Tipa Aprašymo Dokumentacija

7.2.1.1 const_iterator

```
template<typename T>
using Vector< T >::const_iterator = const T*
```

7.2.1.2 const_reverse_iterator

```
template<typename T>
using Vector< T >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

7.2.1.3 iterator

```
template<typename T>
using Vector< T >::iterator = T*
```

7.2.1.4 reverse_iterator

```
template<typename T>
using Vector< T >::reverse_iterator = std::reverse_iterator<iterator>
```

7.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.2.2.1 ~Vector()

```
template<typename T>
Vector< T >::~~Vector () [inline]
```

7.2.2.2 Vector() [1/5]

```
template<typename T>
Vector< T >::Vector () [inline]
```

7.2.2.3 Vector() [2/5]

```
template<typename T>
Vector< T >::Vector (
    size_t n,
    const T & val = T()) [inline]
```

7.2.2.4 Vector() [3/5]

```
template<typename T>
Vector< T >::Vector (
    std::initializer_list< T > il) [inline]
```

7.2.2.5 Vector() [4/5]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & other) [inline]
```

7.2.2.6 Vector() [5/5]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && other) [inline], [noexcept]
```

7.2.3 Metodų Dokumentacija

7.2.3.1 assign() [1/3]

```
template<typename T>
template<typename InputIt>
void Vector< T >::assign (
    InputIt first,
    InputIt last) [inline]
```

7.2.3.2 assign() [2/3]

```
template<typename T>
void Vector< T >::assign (
    size_t count,
    const T & val) [inline]
```

7.2.3.3 assign() [3/3]

```
template<typename T>
void Vector< T >::assign (
    std::initializer_list< T > il) [inline]
```

7.2.3.4 at() [1/2]

```
template<typename T>
T & Vector< T >::at (
    size_t i) [inline]
```

7.2.3.5 at() [2/2]

```
template<typename T>
const T & Vector< T >::at (
    size_t i) const [inline]
```

7.2.3.6 back() [1/2]

```
template<typename T>
T & Vector< T >::back () [inline]
```

7.2.3.7 back() [2/2]

```
template<typename T>
const T & Vector< T >::back () const [inline]
```

7.2.3.8 begin() [1/2]

```
template<typename T>
iterator Vector< T >::begin () [inline]
```

7.2.3.9 begin() [2/2]

```
template<typename T>
const_iterator Vector< T >::begin () const [inline]
```

7.2.3.10 capacity()

```
template<typename T>
size_t Vector< T >::capacity () const [inline]
```

7.2.3.11 cbegin()

```
template<typename T>
const_iterator Vector< T >::cbegin () const [inline]
```

7.2.3.12 cend()

```
template<typename T>
const_iterator Vector< T >::cend () const [inline]
```

7.2.3.13 clear()

```
template<typename T>
void Vector< T >::clear () [inline]
```

7.2.3.14 crbegin()

```
template<typename T>
const_reverse_iterator Vector< T >::crbegin () const [inline]
```

7.2.3.15 crend()

```
template<typename T>
const_reverse_iterator Vector< T >::crend () const [inline]
```

7.2.3.16 data() [1/2]

```
template<typename T>
T * Vector< T >::data () [inline]
```

7.2.3.17 data() [2/2]

```
template<typename T>
const T * Vector< T >::data () const [inline]
```

7.2.3.18 emplace()

```
template<typename T>
template<typename... Args>
iterator Vector< T >::emplace (
    const_iterator pos,
    Args &&... args) [inline]
```

7.2.3.19 emplace_back()

```
template<typename T>
template<typename... Args>
void Vector< T >::emplace_back (
    Args &&... args) [inline]
```

7.2.3.20 empty()

```
template<typename T>
bool Vector< T >::empty () const [inline]
```

7.2.3.21 end() [1/2]

```
template<typename T>
iterator Vector< T >::end () [inline]
```

7.2.3.22 end() [2/2]

```
template<typename T>
const_iterator Vector< T >::end () const [inline]
```

7.2.3.23 erase() [1/2]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator first,
    const_iterator last) [inline]
```

7.2.3.24 erase() [2/2]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator pos) [inline]
```

7.2.3.25 front() [1/2]

```
template<typename T>
T & Vector< T >::front () [inline]
```

7.2.3.26 front() [2/2]

```
template<typename T>
const T & Vector< T >::front () const [inline]
```

7.2.3.27 insert() [1/2]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    const T & val) [inline]
```

7.2.3.28 insert() [2/2]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    size_t count,
    const T & val) [inline]
```

7.2.3.29 max_size()

```
template<typename T>
size_t Vector< T >::max_size () const [inline]
```

7.2.3.30 operator"!="()

```
template<typename T>
bool Vector< T >::operator!= (
    const Vector< T > & other) const [inline]
```

7.2.3.31 operator<()

```
template<typename T>
bool Vector< T >::operator< (
    const Vector< T > & other) const [inline]
```

7.2.3.32 operator<=()

```
template<typename T>
bool Vector< T >::operator<= (
    const Vector< T > & other) const [inline]
```

7.2.3.33 operator=() [1/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    const Vector< T > & other) [inline]
```

7.2.3.34 operator=() [2/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    std::initializer_list< T > il) [inline]
```

7.2.3.35 operator=() [3/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    Vector< T > && other) [inline], [noexcept]
```

7.2.3.36 operator==()

```
template<typename T>
bool Vector< T >::operator== (
    const Vector< T > & other) const [inline]
```

7.2.3.37 operator>()

```
template<typename T>
bool Vector< T >::operator> (
    const Vector< T > & other) const [inline]
```

7.2.3.38 operator>=()

```
template<typename T>
bool Vector< T >::operator>= (
    const Vector< T > & other) const [inline]
```

7.2.3.39 operator[]() [1/2]

```
template<typename T>
T & Vector< T >::operator[] (
    size_t i) [inline]
```

7.2.3.40 operator[]() [2/2]

```
template<typename T>
const T & Vector< T >::operator[] (
    size_t i) const [inline]
```

7.2.3.41 pop_back()

```
template<typename T>
void Vector< T >::pop_back () [inline]
```

7.2.3.42 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
    const T & val) [inline]
```

7.2.3.43 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
    T && val) [inline]
```

7.2.3.44 rbegin() [1/2]

```
template<typename T>
reverse_iterator Vector< T >::rbegin () [inline]
```

7.2.3.45 rbegin() [2/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rbegin () const [inline]
```

7.2.3.46 rend() [1/2]

```
template<typename T>
reverse_iterator Vector< T >::rend () [inline]
```

7.2.3.47 rend() [2/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rend () const [inline]
```

7.2.3.48 reserve()

```
template<typename T>
void Vector< T >::reserve (
    size_t newCap) [inline]
```

7.2.3.49 resize()

```
template<typename T>
void Vector< T >::resize (
    size_t newSize,
    const T & val = T()) [inline]
```

7.2.3.50 shrink_to_fit()

```
template<typename T>
void Vector< T >::shrink_to_fit () [inline]
```

7.2.3.51 size()

```
template<typename T>
size_t Vector< T >::size () const [inline]
```

7.2.3.52 swap()

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & other) [inline], [noexcept]
```

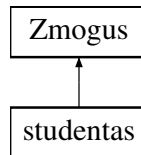
Dokumentacija šiai klasei sugeneruota iš šio failo:

- [src/vector.h](#)

7.3 Zmogus Klasė

```
#include <mylib.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- [Zmogus \(\)](#)
- [Zmogus \(const std::string &v, const std::string &p\)](#)
- virtual [~Zmogus \(\)](#)
- virtual std::string [getVardas \(\)](#) const =0
- virtual std::string [getPavarde \(\)](#) const =0
- virtual void [setVardas \(const std::string &v\)=0](#)
- virtual void [setPavarde \(const std::string &p\)=0](#)
- virtual void [print \(std::ostream &out\) const =0](#)

Apsaugoti Atributai

- std::string [vardas](#)
- std::string [pavarde](#)

Draugai

- std::ostream & [operator<<](#) (std::ostream &out, const [Zmogus](#) &z)

7.3.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.3.1.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [inline]
```

7.3.1.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
    const std::string & v,
    const std::string & p) [inline]
```

7.3.1.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus () [inline], [virtual]
```

7.3.2 Metodų Dokumentacija

7.3.2.1 getPavarde()

```
virtual std::string Zmogus::getPavarde () const [pure virtual]
```

Realizuota [studentas](#).

7.3.2.2 getVardas()

```
virtual std::string Zmogus::getVardas () const [pure virtual]
```

Realizuota [studentas](#).

7.3.2.3 print()

```
virtual void Zmogus::print (  
    std::ostream & out) const [pure virtual]
```

Realizuota [studentas](#).

7.3.2.4 setPavarde()

```
virtual void Zmogus::setPavarde (  
    const std::string & p) [pure virtual]
```

Realizuota [studentas](#).

7.3.2.5 setVardas()

```
virtual void Zmogus::setVardas (  
    const std::string & v) [pure virtual]
```

Realizuota [studentas](#).

7.3.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija

7.3.3.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & out,  
    const Zmogus & z) [friend]
```

7.3.4 Atributų Dokumentacija

7.3.4.1 pavarde

```
std::string Zmogus::pavarde [protected]
```

7.3.4.2 vardas

```
std::string Zmogus::vardas [protected]
```

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [src/mylib.h](#)

skyrius 8

Failo Dokumentacija

8.1 README.md Failo Nuoroda

8.2 src/main.cpp Failo Nuoroda

```
#include <string>
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <cstdlib>
#include <fstream>
#include <sstream>
#include <chrono>
#include "mylib.h"
#include <windows.h>
```

Funkcijos

- int `main` ()

8.2.1 Funkcijos Dokumentacija

8.2.1.1 main()

```
int main ()
```

8.3 test/main.cpp Failo Nuoroda

```
#include <gtest/gtest.h>
#include "mylib.h"
#include <sstream>
```

Funkcijos

- `studentas sukurtiStudenta` (const std::string &v, const std::string &p, std::vector< int > pazymiai, int egz)
- `TEST` (StudentasKonstruktorius, Numatytasis)
- `TEST` (StudentasKonstruktorius, Kopijavimo)
- `TEST` (StudentasKonstruktorius, KopijavimoPreskyrimasOperatorius)
- `TEST` (StudentasKonstruktorius, Perkelimo)
- `TEST` (StudentasKonstruktorius, PerkelimoPreskyrimasOperatorius)
- `TEST` (Skaiciavimai, VidurklisVienas)

- **TEST** (Skaiciavimai, VidurklisKeli)
- **TEST** (Skaiciavimai, VidurklisOhnePazymiu)
- **TEST** (Skaiciavimai, MedianaNelyginisSkacius)
- **TEST** (Skaiciavimai, MedianaLyginisSkacius)
- **TEST** (Skaiciavimai, MedianaOhnePazymiu)
- **TEST** (Skaiciavimai, GalutinisVidurkiu)
- **TEST** (Skaiciavimai, GalutinisMediana)
- **TEST** (Skirstymas, NevykeliaiAtskiriami)
- **TEST** (Skirstymas, VisiVykeliai)
- **TEST** (Skirstymas, VisiNevykeliai)
- **TEST** (Skirstymas, TustiKonteineriaiNesugriuna)
- **TEST** (Isvedimas, PrintRezIsvedasTeisingai)

8.3.1 Funkcijos Dokumentacija

8.3.1.1 sukurtiStudenta()

```
studentas sukurtiStudenta (
    const std::string & v,
    const std::string & p,
    std::vector< int > pazymiai,
    int egz)
```

8.3.1.2 TEST() [1/18]

```
TEST (
    Isvedimas ,
    PrintRezIsvedasTeisingai )
```

8.3.1.3 TEST() [2/18]

```
TEST (
    Skaiciavimai ,
    GalutinisMediana )
```

8.3.1.4 TEST() [3/18]

```
TEST (
    Skaiciavimai ,
    GalutinisVidurkiu )
```

8.3.1.5 TEST() [4/18]

```
TEST (
    Skaiciavimai ,
    MedianaLyginisSkacius )
```

8.3.1.6 TEST() [5/18]

```
TEST (
    Skaiciavimai ,
    MedianaNelyginisSkacius )
```

8.3.1.7 TEST() [6/18]

```
TEST (
    Skaiciavimai ,
    MedianaOhnePazymiu )
```

8.3.1.8 TEST() [7/18]

```
TEST (
    Skaiciavimai ,
    VidurklisKeli )
```

8.3.1.9 TEST() [8/18]

```
TEST (
    Skaiciavimai ,
    VidurklisOhnePazymiu )
```

8.3.1.10 TEST() [9/18]

```
TEST (
    Skaiciavimai ,
    VidurklisVienas )
```

8.3.1.11 TEST() [10/18]

```
TEST (
    Skirstymas ,
    NevykeliaiAtskiriami )
```

8.3.1.12 TEST() [11/18]

```
TEST (
    Skirstymas ,
    TustiKonteineriaiNesugriuna )
```

8.3.1.13 TEST() [12/18]

```
TEST (
    Skirstymas ,
    VisiNevykeliai )
```

8.3.1.14 TEST() [13/18]

```
TEST (
    Skirstymas ,
    VisiVykeliai )
```

8.3.1.15 TEST() [14/18]

```
TEST (
    StudentasKonstruktorius ,
    Kopijavimo )
```

8.3.1.16 TEST() [15/18]

```
TEST (
    StudentasKonstruktorius ,
    KopijavimoPreskyrimasOperatorius )
```

8.3.1.17 TEST() [16/18]

```
TEST (
    StudentasKonstruktorius ,
    Numatytasis )
```

8.3.1.18 TEST() [17/18]

```
TEST (
    StudentasKonstruktorius ,
    Perkelimo )
```

8.3.1.19 TEST() [18/18]

```
TEST (
    StudentasKonstruktorius ,
    PerkelimoPreskyrimasOperatorius )
```

8.4 src/mylib.cpp Failo Nuoroda

```
#include "mylib.h"
#include <iostream>
#include <string>
#include <algorithm>
#include <fstream>
#include <stdexcept>
#include <iomanip>
#include <chrono>
#include <sstream>
```

Funkcijos

- std::string [randomstr](#) ()
- bool [pagalVard](#) (const [studentas](#) &a, const [studentas](#) &b)
- bool [pagalPavard](#) (const [studentas](#) &a, const [studentas](#) &b)
- bool [pagalGal](#) (const [studentas](#) &a, const [studentas](#) &b)
- int [getInt](#) (int min, int max)
- std::string [getFile](#) ()
- void [printRez](#) (std::ostream &out, [Studentai](#)< [studentas](#) > &A, int skaiciavimas)
- void [ivestiRanka](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [generuotiPazymius](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [generuotiViska](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [skaitytilsFailo](#) ([Studentai](#)< [studentas](#) > &A, int &m, std::string &failas)
- void [generuotiFaila](#) ()
- void [skirstymas](#) ([Studentai](#)< [studentas](#) > &studentai, [Studentai](#)< [studentas](#) > &nevykeliai)

8.4.1 Funkcijos Dokumentacija**8.4.1.1 generuotiFaila()**

```
void generuotiFaila ()
```

8.4.1.2 generuotiPazymius()

```
void generuotiPazymius (
    Studentai< studentas > & A,
    int & m)
```

8.4.1.3 generuotiViska()

```
void generuotiViska (
    Studentai< studentas > & A,
    int & m)
```

8.4.1.4 getFile()

```
std::string getFile ()
```

8.4.1.5 getInt()

```
int getInt (
    int min,
    int max)
```

8.4.1.6 iverstiRanka()

```
void iverstiRanka (
    Studentai< studentas > & A,
    int & m)
```

8.4.1.7 pagalGal()

```
bool pagalGal (
    const studentas & a,
    const studentas & b)
```

8.4.1.8 pagalPavard()

```
bool pagalPavard (
    const studentas & a,
    const studentas & b)
```

8.4.1.9 pagalVard()

```
bool pagalVard (
    const studentas & a,
    const studentas & b)
```

8.4.1.10 printRez()

```
void printRez (
    std::ostream & out,
    Studentai< studentas > & A,
    int skaiciavimas)
```

8.4.1.11 randomstr()

```
std::string randomstr ()
```

8.4.1.12 skaitytiIsFailo()

```
void skaitytiIsFailo (
    Studentai< studentas > & A,
    int & m,
    std::string & failas)
```

8.4.1.13 skirstymas()

```
void skirstymas (
    Studentai< studentas > & studentai,
    Studentai< studentas > & nevykeliai)
```

8.5 src/mylib.h Failo Nuoroda

```
#include <string>
#include <vector>
#include <list>
#include <deque>
#include <fstream>
#include <algorithm>
#include <iomanip>
#include "Vector.h"
```

Klasės

- class [Zmogus](#)
- class [studentas](#)

Tipų apibrėžimai

- template<typename T>
using [Studentai](#) = [Vector](#)<T>

Funkcijos

- template<typename Container, typename Comp>
void [rusiuoti](#) (Container &c, Comp comp)
- std::string [randomstr](#) ()
- bool [pagalVard](#) (const [studentas](#) &a, const [studentas](#) &b)
- bool [pagalPavard](#) (const [studentas](#) &a, const [studentas](#) &b)
- bool [pagalGal](#) (const [studentas](#) &a, const [studentas](#) &b)
- int [getInt](#) (int min, int max)
- std::string [getFile](#) ()
- void [printRez](#) (std::ostream &out, [Studentai](#)< [studentas](#) > &A, int skaiciavimas)
- void [ivestiRanka](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [generuotiPazymius](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [generuotiViska](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [skaitytilsFailo](#) ([Studentai](#)< [studentas](#) > &A, int &m, std::string &failas)
- void [generuotiFaila](#) ()
- void [skirstymas](#) ([Studentai](#)< [studentas](#) > &studentai, [Studentai](#)< [studentas](#) > &nevykeliai)

8.5.1 Tipų apibrėžimų Dokumentacija

8.5.1.1 Studentai

```
template<typename T>
using Studentai = Vector<T>
```

8.5.2 Funkcijos Dokumentacija

8.5.2.1 generuotiFaila()

```
void generuotiFaila ()
```

8.5.2.2 generuotiPazymius()

```
void generuotiPazymius (
    Studentai< studentas > & A,
    int & m)
```


8.5.2.3 generuotiViska()

```
void generuotiViska (
    Studentai< studentas > & A,
    int & m)
```

8.5.2.4 getFile()

```
std::string getFile ()
```

8.5.2.5 getInt()

```
int getInt (
    int min,
    int max)
```

8.5.2.6 ivestiRanka()

```
void ivestiRanka (
    Studentai< studentas > & A,
    int & m)
```

8.5.2.7 pagalGal()

```
bool pagalGal (
    const studentas & a,
    const studentas & b)
```

8.5.2.8 pagalPavard()

```
bool pagalPavard (
    const studentas & a,
    const studentas & b)
```

8.5.2.9 pagalVard()

```
bool pagalVard (
    const studentas & a,
    const studentas & b)
```

8.5.2.10 printRez()

```
void printRez (
    std::ostream & out,
    Studentai< studentas > & A,
    int skaiciavimas)
```

8.5.2.11 randomstr()

```
std::string randomstr ()
```

8.5.2.12 rusiuoti()

```
template<typename Container, typename Comp>
void rusiuoti (
    Container & c,
    Comp comp)
```

8.5.2.13 skaitytilsFailo()

```
void skaitytilsFailo (
    Studentai< studentas > & A,
    int & m,
    std::string & failas)
```

8.5.2.14 skirstymas()

```
void skirstymas (
    Studentai< studentas > & studentai,
    Studentai< studentas > & nevykeliai)
```

8.6 mylib.h

Eiti į šio failo dokumentaciją.

```
00001 #pragma once
00002 #include <string>
00003 #include <vector>
00004 #include <list>
00005 #include <deque>
00006 #include <fstream>
00007 #include <algorithm>
00008 #include <iomanip>
00009
00010 #if defined(USE_LIST)
00011     template<typename T> using Studentai = std::list<T>;
00012 #elif defined(USE_DEQUE)
00013     template<typename T> using Studentai = std::deque<T>;
00014 #elif defined(USE_VECTOR)
00015     template<typename T> using Studentai = std::vector<T>;
00016 #else
00017     #include "Vector.h"
00018     template<typename T> using Studentai = Vector<T>;
00019 #endif
00020
00021 class Zmogus {
00022 protected:
00023     std::string vardas;
00024     std::string pavarde;
00025
00026 public:
00027     Zmogus() : vardas(""), pavarde("") {}
00028     Zmogus(const std::string& v, const std::string& p) : vardas(v), pavarde(p) {}
00029
00030     virtual ~Zmogus() {}
00031
00032     virtual std::string getVardas() const = 0;
00033     virtual std::string getPavarde() const = 0;
00034     virtual void setVardas(const std::string& v) = 0;
00035     virtual void setPavarde(const std::string& p) = 0;
00036
00037     virtual void print(std::ostream& out) const = 0;
00038
00039     friend std::ostream& operator<<(std::ostream& out, const Zmogus& z) {
00040         z.print(out);
00041         return out;
00042     }
00043 };
00044
00045 class studentas : public Zmogus {
00046 private:
00047     std::vector<int> paz;
00048     int egz;
00049     double rez;
00050     double gal;
00051
00052 public:
00053     studentas()
00054         : Zmogus(), egz(0), rez(0.0), gal(0.0) {}
00055
00056     studentas(const std::string& v, const std::string& p,
00057         const std::vector<int>& pazymiai, int e)
00058         : Zmogus(v, p), paz(pazymiai), egz(e), rez(0.0), gal(0.0) {}
00059
00060     ~studentas() override {}
00061
00062     studentas(const studentas& other)
00063         : Zmogus(other.vardas, other.pavarde),
```

```

00064         paz(other.paz), egz(other.egz), rez(other.rez), gal(other.gal) {}
00065
00066     studentas& operator=(const studentas& other) {
00067         if (this == &other) return *this;
00068         vardas = other.vardas;
00069         pavarde = other.pavarde;
00070         paz = other.paz;
00071         egz = other.egz;
00072         rez = other.rez;
00073         gal = other.gal;
00074         return *this;
00075     }
00076
00077     studentas(studentas&& other) noexcept
00078         : Zmogus(std::move(other.vardas), std::move(other.pavarde)),
00079         paz(std::move(other.paz)), egz(other.egz), rez(other.rez), gal(other.gal) {
00080         other.vardas = ""; other.pavarde = "";
00081         other.egz = 0; other.rez = 0.0; other.gal = 0.0;
00082     }
00083
00084     studentas& operator=(studentas&& other) noexcept {
00085         if (this == &other) return *this;
00086         vardas = std::move(other.vardas);
00087         pavarde = std::move(other.pavarde);
00088         paz = std::move(other.paz);
00089         egz = other.egz; rez = other.rez; gal = other.gal;
00090         other.vardas = ""; other.pavarde = "";
00091         other.egz = 0; other.rez = 0.0; other.gal = 0.0;
00092         return *this;
00093     }
00094
00095     std::string getVardas() const override { return vardas; }
00096     std::string getPavarde() const override { return pavarde; }
00097     void setVardas(const std::string& v) override { vardas = v; }
00098     void setPavarde(const std::string& p) override { pavarde = p; }
00099
00100     void print(std::ostream& out) const override {
00101         out << std::left << std::setw(15) << vardas
00102             << '\t' << std::setw(15) << pavarde
00103             << '\t' << std::fixed << std::setprecision(2) << gal;
00104     }
00105
00106     std::vector<int> getPaz() const { return paz; }
00107     int getEgz() const { return egz; }
00108     double getRez() const { return rez; }
00109     double getGal() const { return gal; }
00110
00111     void setPaz(const std::vector<int>& p) { paz = p; }
00112     void addPaz(int p) { paz.push_back(p); }
00113     void clearPaz() { paz.clear(); }
00114     void setEgz(int e) { egz = e; }
00115     void setRez(double r) { rez = r; }
00116     void setGal(double g) { gal = g; }
00117
00118     friend std::ostream& operator<<(std::ostream& out, const studentas& s) {
00119         s.print(out);
00120         return out;
00121     }
00122
00123     friend std::istream& operator>>(std::istream& in, studentas& s) {
00124         s.paz.clear();
00125         in >> s.vardas >> s.pavarde;
00126         int x;
00127         while (in >> x) s.paz.push_back(x);
00128         s.egz = s.paz.back();
00129         s.paz.pop_back();
00130         return in;
00131     }
00132
00133     double vid() const {
00134         if (paz.empty()) return 0.0;
00135         double suma = 0;
00136         for (int p : paz) suma += p;
00137         return suma / paz.size();
00138     }
00139
00140     double med() const {
00141         if (paz.empty()) return 0.0;
00142         std::vector<int> sorted = paz;
00143         std::sort(sorted.begin(), sorted.end());
00144         int n = sorted.size();
00145         if (n % 2 == 0) return (sorted[n/2-1] + sorted[n/2]) / 2.0;
00146         else return sorted[n/2];
00147     }
00148 };
00149
00150 template<typename Container, typename Comp>

```

```

00151 void rusiuoti(Container& c, Comp comp) {
00152     #if defined(USE_LIST)
00153         c.sort(comp);
00154     #else
00155         std::sort(c.begin(), c.end(), comp);
00156     #endif
00157 }
00158
00159 std::string randomstr();
00160 bool pagalVard(const studentas& a, const studentas& b);
00161 bool pagalPavard(const studentas& a, const studentas& b);
00162 bool pagalGal(const studentas& a, const studentas& b);
00163 int getInt(int min, int max);
00164 std::string getFile();
00165 void printRez(std::ostream& out, Studentai<studentas>& A, int skaiciavimas);
00166 void ivestiRanka(Studentai<studentas>& A, int& m);
00167 void generuotiPazymius(Studentai<studentas>& A, int& m);
00168 void generuotiViska(Studentai<studentas>& A, int& m);
00169 void skaitytiIsFailo(Studentai<studentas>& A, int& m, std::string& failas);
00170 void generuotiFaila();
00171 void skirstymas(Studentai<studentas>& studentai, Studentai<studentas>& nevykeliai);

```

8.7 src/vector.h Failo Nuoroda

```

#include <stdexcept>
#include <algorithm>
#include <limits>
#include <initializer_list>

```

Klasės

- class [Vector< T >](#)

Funkcijos

- template<typename T>
void [swap](#) ([Vector](#)< T > &a, [Vector](#)< T > &b) noexcept

8.7.1 Funkcijos Dokumentacija

8.7.1.1 swap()

```

template<typename T>
void swap (
    Vector< T > & a,
    Vector< T > & b) [noexcept]

```

8.8 vector.h

Eiti į šio failo dokumentaciją.

```

00001 #pragma once
00002 #include <stdexcept>
00003 #include <algorithm>
00004 #include <limits>
00005 #include <initializer_list>
00006
00007 template <typename T>
00008 class Vector {
00009 private:
00010     T* data_;
00011     size_t size_;
00012     size_t capacity_;
00013
00014     void reallocate(size_t newCap) {
00015         T* newData = new T[newCap];
00016         for (size_t i = 0; i < size_; ++i)
00017             newData[i] = std::move(data_[i]);
00018         delete[] data_;

```

```

00019         data_      = newData;
00020         capacity_ = newCap;
00021     }
00022
00023 public:
00024     // --- Destruttorius ---
00025     ~Vector() { delete[] data_; }
00026
00027     // --- Capacity ---
00028     size_t size() const { return size_; }
00029     size_t capacity() const { return capacity_; }
00030     bool empty() const { return size_ == 0; }
00031
00032     // --- push_back ---
00033     void push_back(const T& val) {
00034         if (size_ == capacity_)
00035             reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00036         data_[size_++] = val;
00037     }
00038
00039     // --- operator[] ---
00040     T& operator[](size_t i) { return data_[i]; }
00041     const T& operator[](size_t i) const { return data_[i]; }
00042
00043     Vector() : data_(nullptr), size_(0), capacity_(0) {}
00044
00045     Vector(size_t n, const T& val = T())
00046         : data_(new T[n]), size_(n), capacity_(n) {
00047         for (size_t i = 0; i < n; ++i)
00048             data_[i] = val;
00049     }
00050
00051     Vector(std::initializer_list<T> il)
00052         : data_(new T[il.size()]), size_(il.size()), capacity_(il.size()) {
00053         size_t i = 0;
00054         for (const auto& v : il)
00055             data_[i++] = v;
00056     }
00057
00058     Vector(const Vector& other)
00059         : data_(new T[other.capacity_]),
00060         size_(other.size_),
00061         capacity_(other.capacity_) {
00062         for (size_t i = 0; i < size_; ++i)
00063             data_[i] = other.data_[i];
00064     }
00065
00066     Vector(Vector&& other) noexcept
00067         : data_(other.data_),
00068         size_(other.size_),
00069         capacity_(other.capacity_) {
00070         other.data_ = nullptr;
00071         other.size_ = 0;
00072         other.capacity_ = 0;
00073     }
00074
00075     Vector& operator=(const Vector& other) {
00076         if (this == &other) return *this;
00077         delete[] data_;
00078         capacity_ = other.capacity_;
00079         size_ = other.size_;
00080         data_ = new T[capacity_];
00081         for (size_t i = 0; i < size_; ++i)
00082             data_[i] = other.data_[i];
00083         return *this;
00084     }
00085
00086     Vector& operator=(Vector&& other) noexcept {
00087         if (this == &other) return *this;
00088         delete[] data_;
00089         data_ = other.data_;
00090         size_ = other.size_;
00091         capacity_ = other.capacity_;
00092         other.data_ = nullptr;
00093         other.size_ = 0;
00094         other.capacity_ = 0;
00095         return *this;
00096     }
00097
00098     Vector& operator=(std::initializer_list<T> il) {
00099         delete[] data_;
00100         size_ = il.size();
00101         capacity_ = il.size();
00102         data_ = new T[capacity_];
00103         size_t i = 0;
00104         for (const auto& v : il)
00105             data_[i++] = v;

```

```

00106         return *this;
00107     }
00108
00109     using iterator          = T*;
00110     using const_iterator    = const T*;
00111     using reverse_iterator  = std::reverse_iterator<iterator>;
00112     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00113
00114     iterator begin()          { return data_; }
00115     iterator end()            { return data_ + size_; }
00116     const_iterator begin()    const { return data_; }
00117     const_iterator end()      const { return data_ + size_; }
00118     const_iterator cbegin()   const { return data_; }
00119     const_iterator cend()     const { return data_ + size_; }
00120
00121     reverse_iterator rbegin() { return reverse_iterator(end()); }
00122     reverse_iterator rend()   { return reverse_iterator(begin()); }
00123     const_reverse_iterator rbegin() const { return const_reverse_iterator(end()); }
00124     const_reverse_iterator rend() const { return const_reverse_iterator(begin()); }
00125     const_reverse_iterator crbegin() const { return const_reverse_iterator(cend()); }
00126     const_reverse_iterator crend() const { return const_reverse_iterator(cbegin()); }
00127
00128     void reserve(size_t newCap) {
00129         if (newCap <= capacity_) return;
00130         reallocate(newCap);
00131     }
00132
00133     void shrink_to_fit() {
00134         if (size_ == capacity_) return;
00135         if (size_ == 0) {
00136             delete[] data_;
00137             data_ = nullptr;
00138             capacity_ = 0;
00139             return;
00140         }
00141         reallocate(size_);
00142     }
00143
00144     size_t max_size() const {
00145         return std::numeric_limits<size_t>::max() / sizeof(T);
00146     }
00147
00148     T& at(size_t i) {
00149         if (i >= size_)
00150             throw std::out_of_range("Vector::at - indeksas " +
00151                                     std::to_string(i) + " >= size " +
00152                                     std::to_string(size_));
00153         return data_[i];
00154     }
00155
00156     const T& at(size_t i) const {
00157         if (i >= size_)
00158             throw std::out_of_range("Vector::at - indeksas " +
00159                                     std::to_string(i) + " >= size " +
00160                                     std::to_string(size_));
00161         return data_[i];
00162     }
00163
00164     T& front() { return data_[0]; }
00165     const T& front() const { return data_[0]; }
00166
00167     T& back() { return data_[size_ - 1]; }
00168     const T& back() const { return data_[size_ - 1]; }
00169
00170     T* data() { return data_; }
00171     const T* data() const { return data_; }
00172
00173     void clear() {
00174         size_ = 0;
00175     }
00176
00177     void pop_back() {
00178         if (size_ > 0) --size_;
00179     }
00180
00181     iterator insert(const_iterator pos, const T& val) {
00182         size_t idx = pos - data_;
00183         if (size_ == capacity_)
00184             reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00185         for (size_t i = size_; i > idx; --i)
00186             data_[i] = std::move(data_[i - 1]);
00187         data_[idx] = val;
00188         ++size_;
00189         return data_ + idx;
00190     }
00191
00192

```

```

00193     iterator insert(const_iterator pos, size_t count, const T& val) {
00194         size_t idx = pos - data_;
00195         if (size_ + count > capacity_)
00196             reallocate(std::max(size_ + count, capacity_ * 2));
00197         for (size_t i = size_ + count - 1; i >= idx + count; --i)
00198             data_[i] = std::move(data_[i - count]);
00199         for (size_t i = 0; i < count; ++i)
00200             data_[idx + i] = val;
00201         size_ += count;
00202         return data_ + idx;
00203     }
00204
00205     iterator erase(const_iterator pos) {
00206         size_t idx = pos - data_;
00207         for (size_t i = idx; i < size_ - 1; ++i)
00208             data_[i] = std::move(data_[i + 1]);
00209         --size_;
00210         return data_ + idx;
00211     }
00212
00213     iterator erase(const_iterator first, const_iterator last) {
00214         size_t idxFirst = first - data_;
00215         size_t idxLast = last - data_;
00216         size_t count = idxLast - idxFirst;
00217         for (size_t i = idxFirst; i < size_ - count; ++i)
00218             data_[i] = std::move(data_[i + count]);
00219         size_ -= count;
00220         return data_ + idxFirst;
00221     }
00222
00223     void resize(size_t newSize, const T& val = T()) {
00224         if (newSize > capacity_)
00225             reallocate(newSize);
00226         if (newSize > size_)
00227             for (size_t i = size_; i < newSize; ++i)
00228                 data_[i] = val;
00229         size_ = newSize;
00230     }
00231
00232     template <typename... Args>
00233     iterator emplace(const_iterator pos, Args&&... args) {
00234         size_t idx = pos - data_;
00235         if (size_ == capacity_)
00236             reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00237         for (size_t i = size_; i > idx; --i)
00238             data_[i] = std::move(data_[i - 1]);
00239         data_[idx] = T(std::forward<Args>(args)...);
00240         ++size_;
00241         return data_ + idx;
00242     }
00243
00244     template <typename... Args>
00245     void emplace_back(Args&&... args) {
00246         if (size_ == capacity_)
00247             reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00248         data_[size_++] = T(std::forward<Args>(args)...);
00249     }
00250
00251     void push_back(T&& val) {
00252         if (size_ == capacity_)
00253             reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00254         data_[size_++] = std::move(val);
00255     }
00256
00257     void assign(size_t count, const T& val) {
00258         clear();
00259         resize(count, val);
00260     }
00261
00262     void assign(std::initializer_list<T> il) {
00263         *this = il;
00264     }
00265
00266     void swap(Vector& other) noexcept {
00267         std::swap(data_, other.data_);
00268         std::swap(size_, other.size_);
00269         std::swap(capacity_, other.capacity_);
00270     }
00271
00272     bool operator==(const Vector& other) const {
00273         if (size_ != other.size_) return false;
00274         for (size_t i = 0; i < size_; ++i)
00275             if (data_[i] != other.data_[i]) return false;
00276         return true;
00277     }
00278
00279     bool operator!=(const Vector& other) const { return !(*this == other); }

```

```

00280
00281     bool operator<(const Vector& other) const {
00282         return std::lexicographical_compare(begin(), end(),
00283                                             other.begin(), other.end());
00284     }
00285
00286     bool operator<=(const Vector& other) const { return !(other < *this); }
00287     bool operator>(const Vector& other) const { return other < *this; }
00288     bool operator>=(const Vector& other) const { return !(*this < other); }
00289
00290     template <typename InputIt>
00291     void assign(InputIt first, InputIt last) {
00292         clear();
00293         for (auto it = first; it != last; ++it)
00294             push_back(*it);
00295     }
00296 };
00297
00298 template <typename T>
00299 void swap(Vector<T>& a, Vector<T>& b) noexcept {
00300     a.swap(b);
00301 }

```

8.9 test/benchmark.cpp Failo Nuoroda

```

#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip>
#include "Vector.h"

```

Tipų apibrėžimai

- using `ms` = `std::chrono::duration<double, std::milli>`

Funkcijos

- template<typename Container>
double `benchmark_push_back` (unsigned int sz)
- template<typename Container>
int `count_reallocations` (unsigned int sz)
- int `main` ()

8.9.1 Tipų apibrėžimų Dokumentacija

8.9.1.1 ms

```
using ms = std::chrono::duration<double, std::milli>
```

8.9.2 Funkcijos Dokumentacija

8.9.2.1 benchmark_push_back()

```

template<typename Container>
double benchmark_push_back (
    unsigned int sz)

```

8.9.2.2 count_reallocations()

```

template<typename Container>
int count_reallocations (
    unsigned int sz)

```


8.9.2.3 main()

```
int main ()
```

8.10 test/benchmark_full.cpp Failo Nuoroda

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <string>
#include "Vector.h"
#include "mylib.h"
```

Funkcijos

- auto `ms` (auto d)
- template<typename Container>
void `skaityti` (Container &A, const std::string &failas)
- template<typename Container>
void `skaiciuoti` (Container &A)
- template<typename Container>
void `rusiuoti_bench` (Container &A)
- template<typename Container>
void `skirstyti_bench` (Container &studentai, Container &nevykeliai)
- template<typename Container>
void `benchmark` (const std::string &failas, const std::string &label)
- int `main` ()

8.10.1 Funkcijos Dokumentacija

8.10.1.1 benchmark()

```
template<typename Container>
void benchmark (
    const std::string & failas,
    const std::string & label)
```

8.10.1.2 main()

```
int main ()
```

8.10.1.3 ms()

```
auto ms (
    auto d)
```

8.10.1.4 rusiuoti_bench()

```
template<typename Container>
void rusiuoti_bench (
    Container & A)
```

8.10.1.5 skaiciuoti()

```
template<typename Container>
void skaiciuoti (
    Container & A)
```

8.10.1.6 skaityti()

```
template<typename Container>
void skaityti (
    Container & A,
    const std::string & failas)
```

8.10.1.7 skirstyti_bench()

```
template<typename Container>
void skirstyti_bench (
    Container & studentai,
    Container & nevykeliai)
```

8.11 test/vector_test.cpp Failo Nuoroda

```
#include <gtest/gtest.h>
#include "Vector.h"
```

Funkcijos

- [TEST](#) (VectorKonstruktoriai, Tuščias)
- [TEST](#) (VectorKonstruktoriai, SuDydžiu)
- [TEST](#) (VectorKonstruktoriai, InitializerList)
- [TEST](#) (VectorKonstruktoriai, Copy)
- [TEST](#) (VectorKonstruktoriai, Move)
- [TEST](#) (VectorKonstruktoriai, CopyOperator)
- [TEST](#) (VectorKonstruktoriai, MoveOperator)
- [TEST](#) (VectorCapacity, Reserve)
- [TEST](#) (VectorCapacity, ReserveNemažinaCapacity)
- [TEST](#) (VectorCapacity, ShrinkToFit)
- [TEST](#) (VectorCapacity, ShrinkToFitTuščias)
- [TEST](#) (VectorAccess, OperatorBracket)
- [TEST](#) (VectorAccess, At)
- [TEST](#) (VectorAccess, AtIšmetaException)
- [TEST](#) (VectorAccess, FrontBack)
- [TEST](#) (VectorAccess, Data)
- [TEST](#) (VectorModifiers, PushBack)
- [TEST](#) (VectorModifiers, PopBack)
- [TEST](#) (VectorModifiers, Clear)
- [TEST](#) (VectorModifiers, InsertViduryje)
- [TEST](#) (VectorModifiers, EraseVienas)
- [TEST](#) (VectorModifiers, EraseRangas)
- [TEST](#) (VectorModifiers, Resize)
- [TEST](#) (VectorModifiers, EmplaceBack)
- [TEST](#) (VectorModifiers, Swap)
- [TEST](#) (VectorIteratoriai, RangeFor)
- [TEST](#) (VectorIteratoriai, StdSort)
- [TEST](#) (VectorIteratoriai, StdFind)

- [TEST](#) (VectorIteratorai, Cbegin)
- [TEST](#) (VectorOperatoriai, Lygybė)
- [TEST](#) (VectorOperatoriai, Nelygybė)
- [TEST](#) (VectorOperatoriai, MažiauUž)
- [TEST](#) (VectorCapacity, DoubleStrategy)

8.11.1 Funkcijos Dokumentacija

8.11.1.1 TEST() [1/33]

```
TEST (
    VectorAccess ,
    At )
```

8.11.1.2 TEST() [2/33]

```
TEST (
    VectorAccess ,
    AtIšmetaException )
```

8.11.1.3 TEST() [3/33]

```
TEST (
    VectorAccess ,
    Data )
```

8.11.1.4 TEST() [4/33]

```
TEST (
    VectorAccess ,
    FrontBack )
```

8.11.1.5 TEST() [5/33]

```
TEST (
    VectorAccess ,
    OperatorBracket )
```

8.11.1.6 TEST() [6/33]

```
TEST (
    VectorCapacity ,
    DoubleStrategy )
```

8.11.1.7 TEST() [7/33]

```
TEST (
    VectorCapacity ,
    Reserve )
```

8.11.1.8 TEST() [8/33]

```
TEST (
    VectorCapacity ,
    ReserveNemažinaCapacity )
```

8.11.1.9 TEST() [9/33]

```
TEST (
    VectorCapacity ,
    ShrinkToFit )
```

8.11.1.10 TEST() [10/33]

```
TEST (
    VectorCapacity ,
    ShrinkToFitTuščias )
```

8.11.1.11 TEST() [11/33]

```
TEST (
    VectorIteratoriai ,
    Cbegin )
```

8.11.1.12 TEST() [12/33]

```
TEST (
    VectorIteratoriai ,
    RangeFor )
```

8.11.1.13 TEST() [13/33]

```
TEST (
    VectorIteratoriai ,
    StdFind )
```

8.11.1.14 TEST() [14/33]

```
TEST (
    VectorIteratoriai ,
    StdSort )
```

8.11.1.15 TEST() [15/33]

```
TEST (
    VectorKonstruktoriai ,
    Copy )
```

8.11.1.16 TEST() [16/33]

```
TEST (
    VectorKonstruktoriai ,
    CopyOperator )
```

8.11.1.17 TEST() [17/33]

```
TEST (
    VectorKonstruktoriai ,
    InitializerList )
```

8.11.1.18 TEST() [18/33]

```
TEST (
    VectorKonstruktoriai ,
    Move )
```

8.11.1.19 TEST() [19/33]

```
TEST (
    VectorKonstruktoriai ,
    MoveOperator )
```

8.11.1.20 TEST() [20/33]

```
TEST (
    VectorKonstruktoriai ,
    SuDydžiu )
```

8.11.1.21 TEST() [21/33]

```
TEST (
    VectorKonstruktoriai ,
    Tuščias )
```

8.11.1.22 TEST() [22/33]

```
TEST (
    VectorModifiers ,
    Clear )
```

8.11.1.23 TEST() [23/33]

```
TEST (
    VectorModifiers ,
    EmplaceBack )
```

8.11.1.24 TEST() [24/33]

```
TEST (
    VectorModifiers ,
    EraseRangas )
```

8.11.1.25 TEST() [25/33]

```
TEST (
    VectorModifiers ,
    EraseVienas )
```

8.11.1.26 TEST() [26/33]

```
TEST (
    VectorModifiers ,
    InsertViduryje )
```

8.11.1.27 TEST() [27/33]

```
TEST (
    VectorModifiers ,
    PopBack )
```

8.11.1.28 TEST() [28/33]

```
TEST (
    VectorModifiers ,
    PushBack )
```

8.11.1.29 TEST() [29/33]

```
TEST (
    VectorModifiers ,
    Resize )
```

8.11.1.30 TEST() [30/33]

```
TEST (
    VectorModifiers ,
    Swap )
```

8.11.1.31 TEST() [31/33]

```
TEST (
    VectorOperatoriai ,
    Lygybė )
```

8.11.1.32 TEST() [32/33]

```
TEST (
    VectorOperatoriai ,
    MažiauUž )
```

8.11.1.33 TEST() [33/33]

```
TEST (
    VectorOperatoriai ,
    Nelygybė )
```

Rodyklë

- ~Vector
 - Vector< T >, [24](#)
- ~Zmogus
 - Zmogus, [30](#)
- ~studentas
 - studentas, [20](#)
- addPaz
 - studentas, [20](#)
- assign
 - Vector< T >, [25](#)
- at
 - Vector< T >, [25](#)
- back
 - Vector< T >, [25](#)
- begin
 - Vector< T >, [25](#)
- benchmark
 - benchmark_full.cpp, [47](#)
- benchmark.cpp
 - benchmark_push_back, [46](#)
 - count_reallocations, [46](#)
 - main, [46](#)
 - ms, [46](#)
- benchmark_full.cpp
 - benchmark, [47](#)
 - main, [47](#)
 - ms, [47](#)
 - rusiuoti_bench, [47](#)
 - skaiciuoti, [47](#)
 - skaityti, [48](#)
 - skirstyti_bench, [48](#)
- benchmark_push_back
 - benchmark.cpp, [46](#)
- capacity
 - Vector< T >, [25](#)
- cbegin
 - Vector< T >, [25](#)
- cend
 - Vector< T >, [26](#)
- clear
 - Vector< T >, [26](#)
- clearPaz
 - studentas, [20](#)
- const_iterator
 - Vector< T >, [24](#)
- const_reverse_iterator
 - Vector< T >, [24](#)
- count_reallocations
 - benchmark.cpp, [46](#)
- crbegin
 - Vector< T >, [26](#)
- crend
 - Vector< T >, [26](#)
- data
 - Vector< T >, [26](#)
- emplace
 - Vector< T >, [26](#)
- emplace_back
 - Vector< T >, [26](#)
- empty
 - Vector< T >, [26](#)
- end
 - Vector< T >, [26](#)
- erase
 - Vector< T >, [27](#)
- front
 - Vector< T >, [27](#)
- generuotiFaila
 - mylib.cpp, [36](#)
 - mylib.h, [38](#)
- generuotiPazymius
 - mylib.cpp, [36](#)
 - mylib.h, [38](#)
- generuotiViska
 - mylib.cpp, [36](#)
 - mylib.h, [38](#)
- getEgz
 - studentas, [20](#)
- getFile
 - mylib.cpp, [36](#)
 - mylib.h, [39](#)
- getGal
 - studentas, [20](#)
- getInt
 - mylib.cpp, [37](#)
 - mylib.h, [39](#)
- getPavarde
 - studentas, [21](#)
 - Zmogus, [30](#)
- getPaz
 - studentas, [21](#)
- getRez
 - studentas, [21](#)

- getVardas
 - studentas, [21](#)
 - Zmogus, [30](#)
- insert
 - Vector< T >, [27](#)
- iterator
 - Vector< T >, [24](#)
- ivestiRanka
 - mylib.cpp, [37](#)
 - mylib.h, [39](#)
- main
 - benchmark.cpp, [46](#)
 - benchmark_full.cpp, [47](#)
 - main.cpp, [33](#)
- main.cpp
 - main, [33](#)
 - sukurtiStudenta, [34](#)
 - TEST, [34–36](#)
- max_size
 - Vector< T >, [27](#)
- med
 - studentas, [21](#)
- ms
 - benchmark.cpp, [46](#)
 - benchmark_full.cpp, [47](#)
- mylib.cpp
 - generuotiFaila, [36](#)
 - generuotiPazymius, [36](#)
 - generuotiViska, [36](#)
 - getFile, [36](#)
 - getInt, [37](#)
 - ivestiRanka, [37](#)
 - pagalGal, [37](#)
 - pagalPavard, [37](#)
 - pagalVard, [37](#)
 - printRez, [37](#)
 - randomstr, [37](#)
 - skaitytilsFailo, [37](#)
 - skirstymas, [37](#)
- mylib.h
 - generuotiFaila, [38](#)
 - generuotiPazymius, [38](#)
 - generuotiViska, [38](#)
 - getFile, [39](#)
 - getInt, [39](#)
 - ivestiRanka, [39](#)
 - pagalGal, [39](#)
 - pagalPavard, [39](#)
 - pagalVard, [39](#)
 - printRez, [39](#)
 - randomstr, [39](#)
 - rusiuoti, [39](#)
 - skaitytilsFailo, [39](#)
 - skirstymas, [40](#)
 - Studentai, [38](#)
- ObjektinisProg, [1](#)
- operator!=
 - Vector< T >, [27](#)
- operator<
 - Vector< T >, [27](#)
- operator<<
 - studentas, [22](#)
 - Zmogus, [31](#)
- operator<=
 - Vector< T >, [27](#)
- operator>
 - Vector< T >, [28](#)
- operator>>
 - studentas, [22](#)
- operator>=
 - Vector< T >, [28](#)
- operator=
 - studentas, [21](#)
 - Vector< T >, [28](#)
- operator==
 - Vector< T >, [28](#)
- operator[]
 - Vector< T >, [28](#)
- pagalGal
 - mylib.cpp, [37](#)
 - mylib.h, [39](#)
- pagalPavard
 - mylib.cpp, [37](#)
 - mylib.h, [39](#)
- pagalVard
 - mylib.cpp, [37](#)
 - mylib.h, [39](#)
- pavarde
 - Zmogus, [31](#)
- pop_back
 - Vector< T >, [28](#)
- print
 - studentas, [21](#)
 - Zmogus, [31](#)
- printRez
 - mylib.cpp, [37](#)
 - mylib.h, [39](#)
- push_back
 - Vector< T >, [28, 29](#)
- randomstr
 - mylib.cpp, [37](#)
 - mylib.h, [39](#)
- rbegin
 - Vector< T >, [29](#)
- README.md, [33](#)
- rend
 - Vector< T >, [29](#)
- reserve
 - Vector< T >, [29](#)
- resize
 - Vector< T >, [29](#)
- reverse_iterator
 - Vector< T >, [24](#)

- rusiuoti
 - mylib.h, 39
- rusiuoti_bench
 - benchmark_full.cpp, 47
- setEgz
 - studentas, 21
- setGal
 - studentas, 21
- setPavarde
 - studentas, 21
 - Zmogus, 31
- setPaz
 - studentas, 21
- setRez
 - studentas, 22
- setVardas
 - studentas, 22
 - Zmogus, 31
- shrink_to_fit
 - Vector< T >, 29
- size
 - Vector< T >, 29
- skaiciuoti
 - benchmark_full.cpp, 47
- skaityti
 - benchmark_full.cpp, 48
- skaitytisFailo
 - mylib.cpp, 37
 - mylib.h, 39
- skirstymas
 - mylib.cpp, 37
 - mylib.h, 40
- skirstyti_bench
 - benchmark_full.cpp, 48
- src Direktorijos aprašas, 17
- src/main.cpp, 33
- src/mylib.cpp, 36
- src/mylib.h, 38, 40
- src/vector.h, 42
- Studentai
 - mylib.h, 38
- studentas, 19
 - ~studentas, 20
 - addPaz, 20
 - clearPaz, 20
 - getEgz, 20
 - getGal, 20
 - getPavarde, 21
 - getPaz, 21
 - getRez, 21
 - getVardas, 21
 - med, 21
 - operator<<, 22
 - operator>>, 22
 - operator=, 21
 - print, 21
 - setEgz, 21
 - setGal, 21
 - setPavarde, 21
 - setPaz, 21
 - setRez, 22
 - setVardas, 22
 - studentas, 20
 - vid, 22
- sukurtiStudenta
 - main.cpp, 34
- swap
 - Vector< T >, 29
 - vector.h, 42
- TEST
 - main.cpp, 34–36
 - vector_test.cpp, 49–52
- test Direktorijos aprašas, 17
- test/benchmark.cpp, 46
- test/benchmark_full.cpp, 47
- test/main.cpp, 33
- test/vector_test.cpp, 48
- vardas
 - Zmogus, 31
- Vector
 - Vector< T >, 24
- Vector< T >, 22
 - ~Vector, 24
 - assign, 25
 - at, 25
 - back, 25
 - begin, 25
 - capacity, 25
 - cbegin, 25
 - cend, 26
 - clear, 26
 - const_iterator, 24
 - const_reverse_iterator, 24
 - crbegin, 26
 - crend, 26
 - data, 26
 - emplace, 26
 - emplace_back, 26
 - empty, 26
 - end, 26
 - erase, 27
 - front, 27
 - insert, 27
 - iterator, 24
 - max_size, 27
 - operator!=, 27
 - operator<, 27
 - operator<=, 27
 - operator>, 28
 - operator>=, 28
 - operator=, 28
 - operator==, 28
 - operator[], 28
 - pop_back, 28
 - push_back, 28, 29

- [rbegin](#), [29](#)
 - [rend](#), [29](#)
 - [reserve](#), [29](#)
 - [resize](#), [29](#)
 - [reverse_iterator](#), [24](#)
 - [shrink_to_fit](#), [29](#)
 - [size](#), [29](#)
 - [swap](#), [29](#)
 - [Vector](#), [24](#)
- [vector.h](#)
 - [swap](#), [42](#)
- [vector_test.cpp](#)
 - [TEST](#), [49–52](#)
- [vid](#)
 - [studentas](#), [22](#)
- [Zmogus](#), [30](#)
 - [~Zmogus](#), [30](#)
 - [getPavarde](#), [30](#)
 - [getVardas](#), [30](#)
 - [operator<<](#), [31](#)
 - [pavarde](#), [31](#)
 - [print](#), [31](#)
 - [setPavarde](#), [31](#)
 - [setVardas](#), [31](#)
 - [vardas](#), [31](#)
 - [Zmogus](#), [30](#)